

UNIT-4

Machine Learning

Recurrent Neural Network 2024

A Recurrent Neural Network (RNN) is the type of artificial neural network specially designed to process sequential data.

Unlike traditional neural networks RNN can remember previous information and use it to make decisions about current inputs.

In simple words, RNN has memory, which helps it understand the relationship between past and present data.

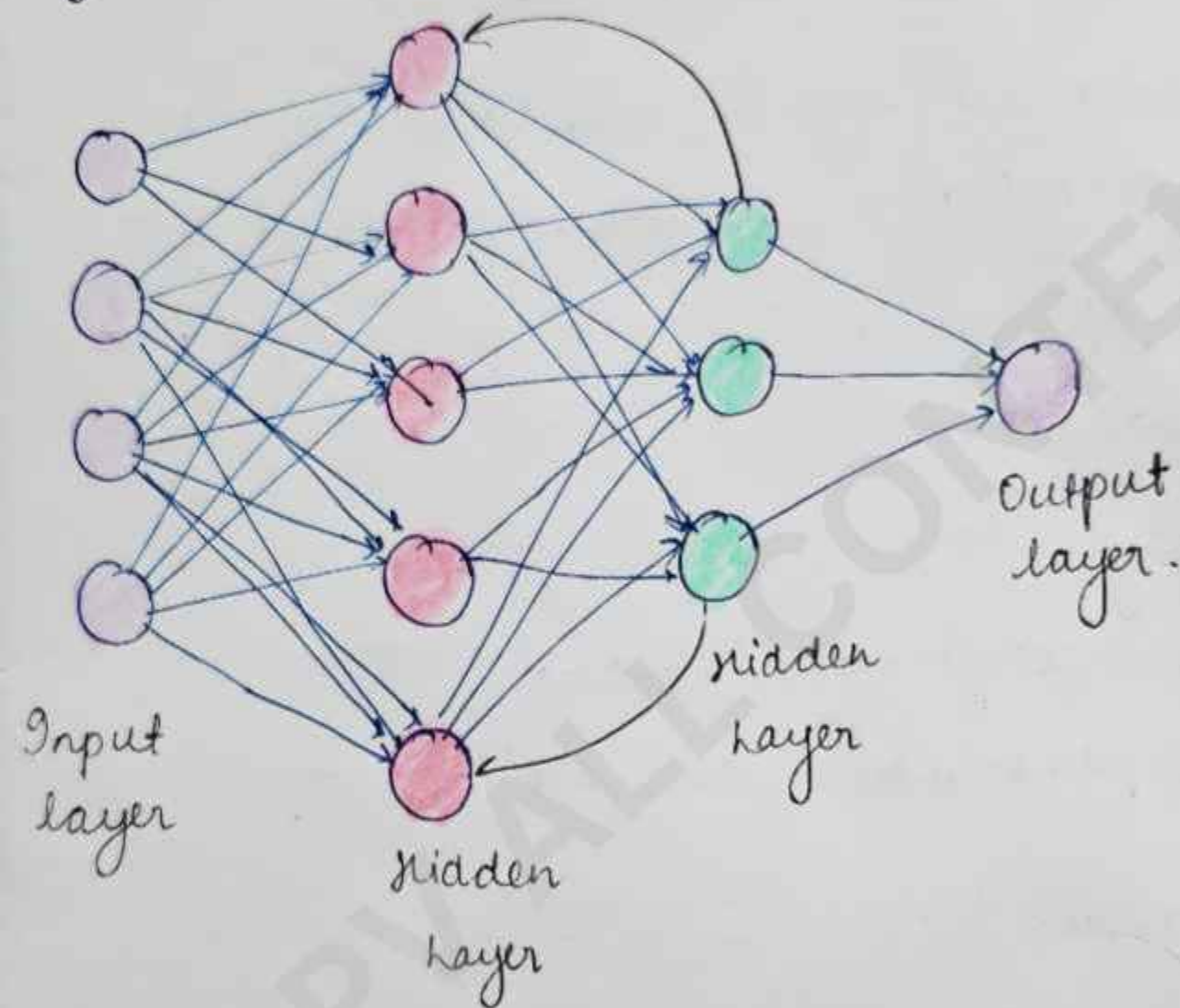
Need of RNN

Normal neural networks process each input independently and cannot remember previous information.

But many tasks require memory, such as.

- Language translation
- Speech Recognition.
- Text Predictions

RNN solves this problem by storing information from previous steps.



Recurrent Neural
Network.

Example → Consider the sentence:-

"The movie was not good".

To understand the word "good", the model should remember the earlier word "not".

RNNs use previous context to make better predictions.

Advantages

- Handles sequential data
- Shares parameters across time steps

Limitations.

- Suffer from vanishing and exploding gradients during training.
- Difficult to learn long-term dependencies.

To overcome RNN limitations, advanced architectures were developed :-

① LSTM (Long-Short Term Memory)

- Uses memory cells and gates
- Better at learning long-term dependencies.

② GRU (Gated Recurrent Unit)

- Simpler than LSTM.

- Faster training with similar performance

Long Short-Term Memory

2024, 2025

LSTM (Long Short-Term Memory) is a special type of Recurrent Neural Network (RNN) designed to remember information for a long period of time.

It was developed to solve the vanishing gradient problem of traditional RNNs.

Why LSTM is Needed?

In a simple RNN, information from earlier inputs may be forgotten after many time steps.

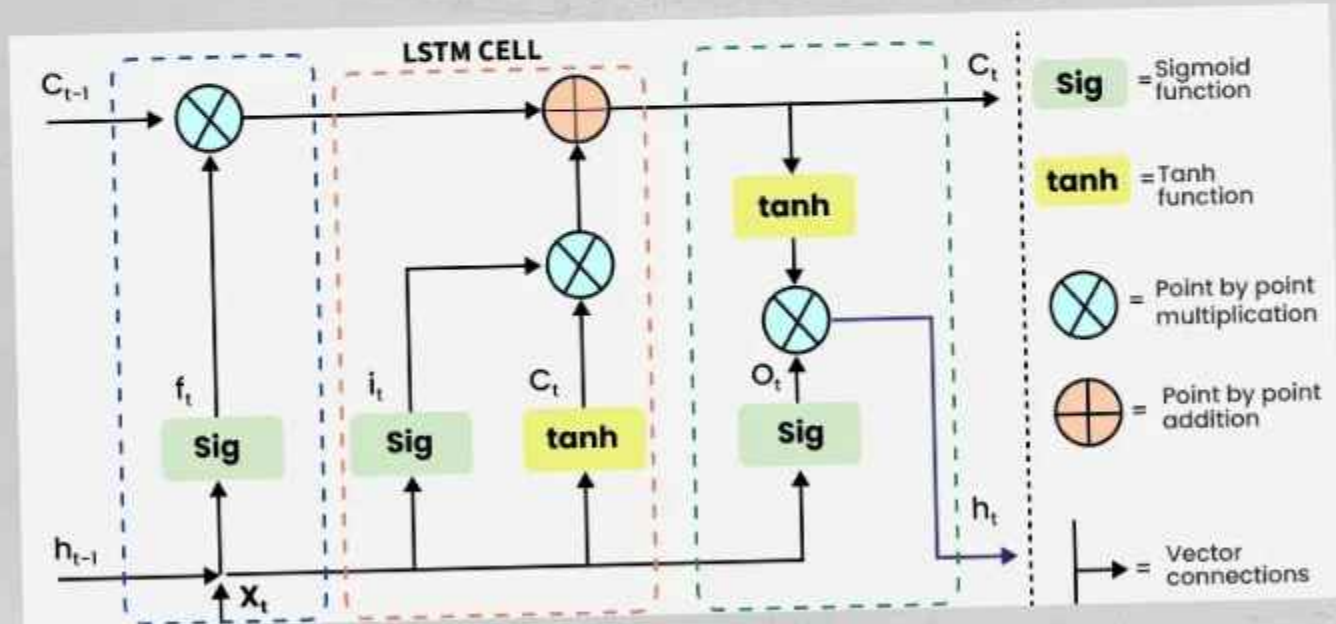
Example

→ "I grew up in France. I speak fluent French."

To understand why "French" is used, the model should remember "France" from earlier in the sentence.

RNNs often struggle with such long-term dependencies, while LSTMs can remember them.

Structure of LSTM.



An LSTM contains a cell state (memory) and three gates.

- ① Forget Gate - Decides what information should be removed from memory.
- ② Input Gate - Decides what information should be removed from memory.
- ③ Output Gate - Decides what information should be ~~removed~~ sent as output.

Working of LSTM.

Step 1 -> Removes unnecessary information from the cell state

Step 2 - Add useful new information to the cell state.

Step 3 - Update the memory by combining old and new information.

Step 4. Produce the final output and hidden state.

Advantages

- Remembers information for a long time
- Solve vanishing gradient problem.
- Good for sequential data.

- Better performance than simple RNN:

Disadvantages.

- More complex than RNN
- Requires more computation and memory.
- Training can be slower.

Gated Recurrent Unit

A Gated Recurrent Unit (GRU) is a type of Recurrent neural network (RNN) architecture designed to handle sequential data and mitigate the vanishing gradient problem that affects traditional RNNs.

- GRUs use gates to control how information flows through the network, allowing them to remember long-term dependencies more effectively.

Components of a GRU

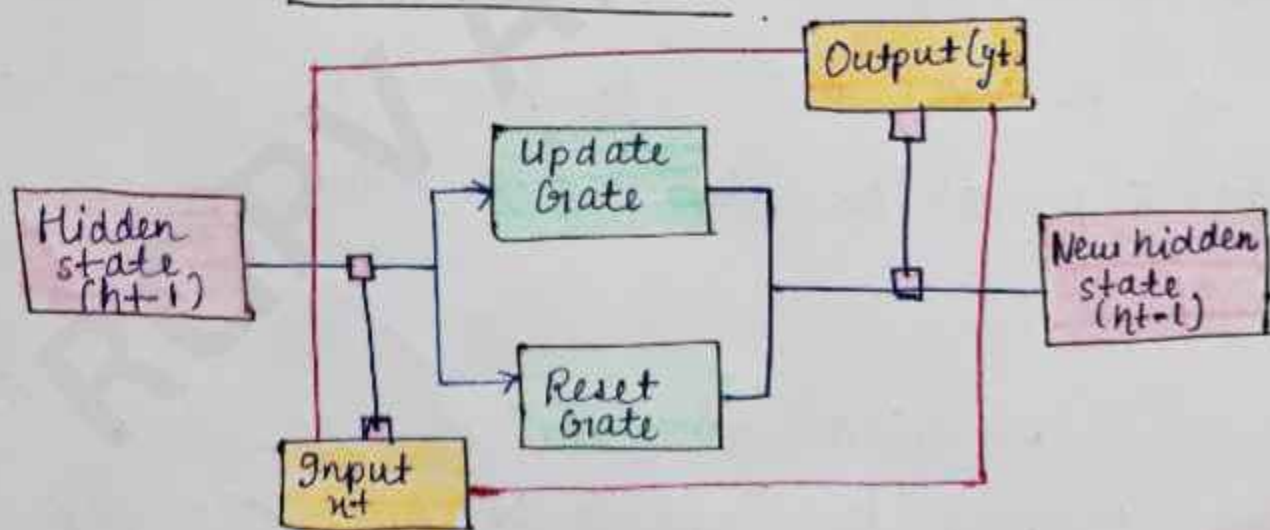
A GRU has two gates:-

1. Update Gate (z_t) • Determines how much of the previous hidden state should be retained.

• Acts similarly to the combination of the forget and input gates in an LSTM

2. Reset Gate (r_t) • Controls how much past information to forget when computing the new candidate state.

Structure



Advantages

- Simpler Architecture than LSTM.
- Fewer parameters, reducing computation and memory requirements.
- Effective for sequence modeling tasks such as:
 - Natural Language Processing (NLP)
 - Speech Recognition
 - Time-series forecasting
 - Machine Translation

Attention Model

The Attention model is a mechanism that helps a neural network focus on the most important parts on the input while generating an output.

It was introduced to overcome the limitations of traditional Recurrent Neural Network (RNNs) and Encoder - Decoder models, which struggle with long sequence

Why Attention model is needed.

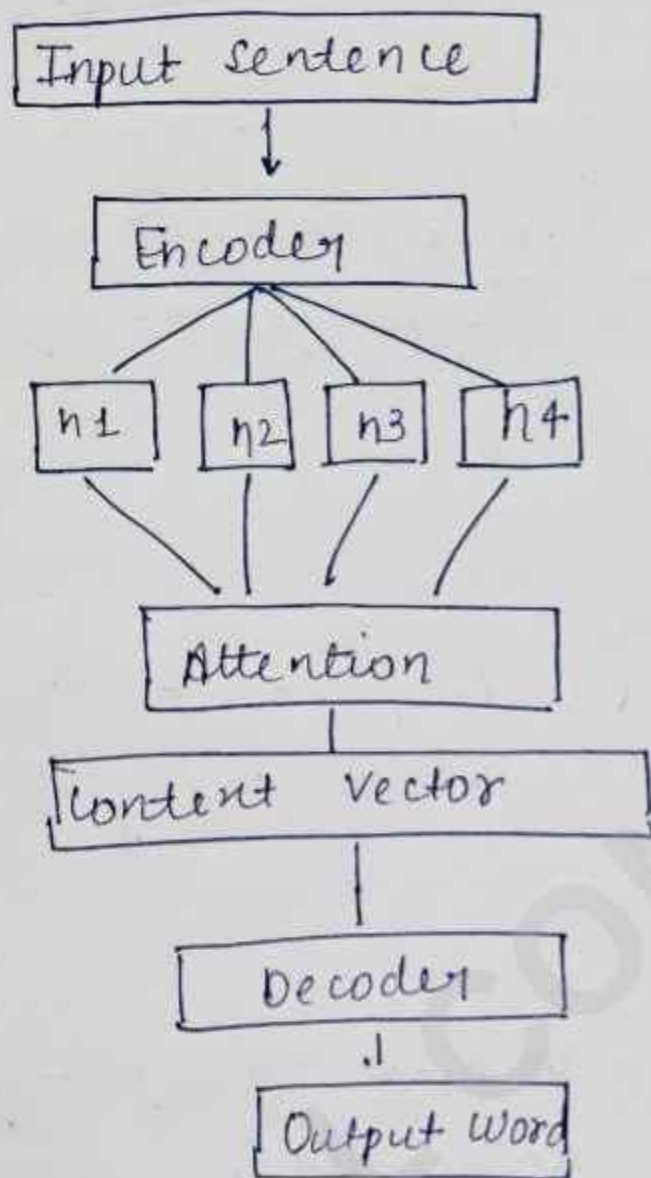
In a basic Encoder-Decoder model:-

- The encoder converts the entire input sentence into a single fixed-size vector.
 - The decoder uses this vector to generate the output.
- For long sentences, important information may be lost because everything is compressed into one vector.

Example

Input → "The book that I bought yesterday is very interesting."

- When translating this sentence, the model should pay more attention to words like book, bought, and interesting at different decoding steps.
- Attention allows the model to look back at all encoder outputs instead of relying on only one vector.



Simple Diagram of
Attention model.

where h_1, h_2, h_3, h_4 are hidden states.

Advantages.

- 1 Handles long sentence better
- 2 Improves translation accuracy
- 3 Focuses on relevant words.
- 4 Reduces information loss.
- 5 Forms the foundation of the Transformer architecture.

Short Definition

→ Attention model is a mechanism that enables a neural network to dynamically focus on the most relevant parts of the input sequence while generating each output, improving performance on long-sequence tasks.

Markov Decision Process (MDP)

A Markov Decision Process (MDP) is a ²⁰²⁵~~2024~~ mathematical framework used in Reinforcement Learning (RL) to model decision-making problems where outcomes depend on the current state and action taken.

Markov Property:-

The future depends only on the current state, not on the sequence of past states.

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, S_2, \dots, S_t)$$

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, S_2, \dots, S_t)$$

This is called Markov Property.

Components of MDP

An MDP is represented as:-

(S, A, P, R, γ)

1. State (S) :- Set of all possible state of the environment.

Example:- Position in a maze

2. Action (A) :- Set of action available to the agent.

Example:- Move up, Down, left, Right

3. Transition Probability (P) :- Probability of moving from one state to another after taking an action.

$$P(s' | s, a)$$

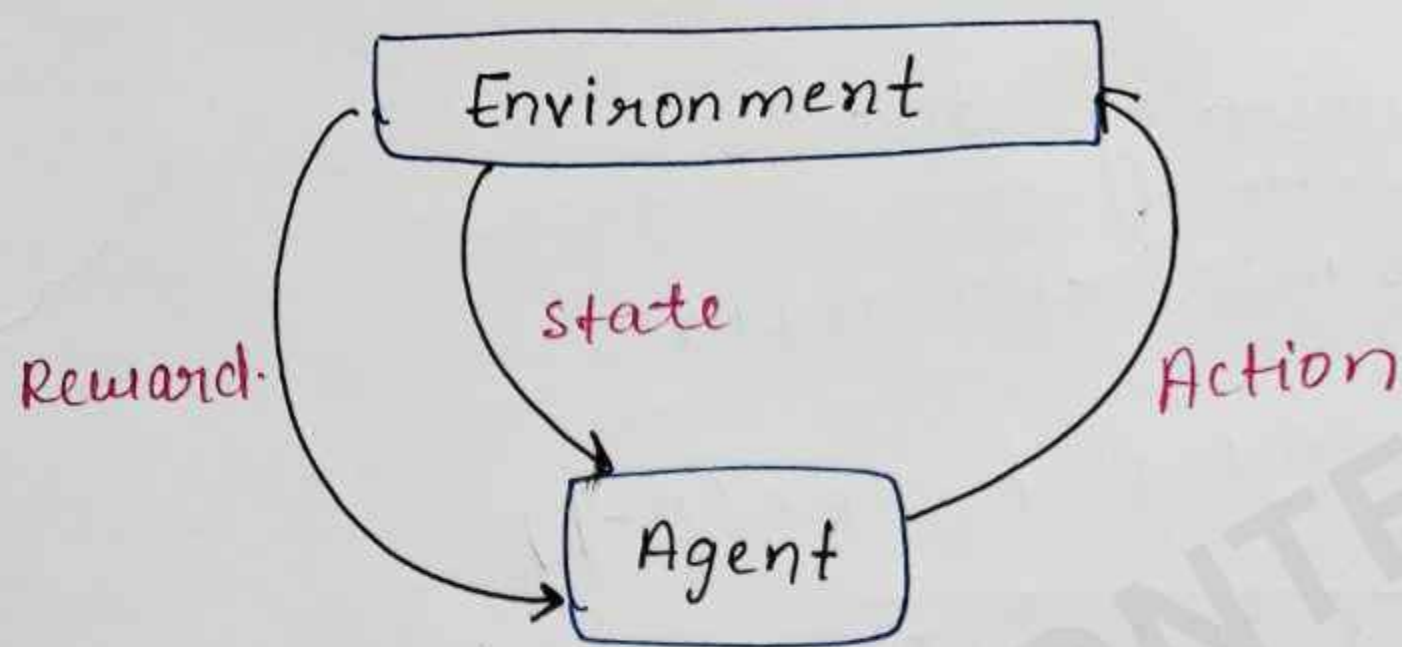
4. Reward Function (R) :- Reward received after performing an action

$$R(s, a)$$

5. Discount Factor (γ) :- Determines the Importance of future rewards

$$0 \leq \gamma \leq 1$$

- $\gamma = 0 \rightarrow$ only immediate reward matter
- γ close to 1 $\rightarrow$ Future rewards are important.



Markov Decision Process

Importance of MDP

- Foundation of Reinforcement Learning.
- Models sequential decision-making.
- Used in Robotics, game playing, recommendation systems, and self-driving cars.

Reinforcement Learning.

Reinforcement learning is a type of machine learning in which an agent learns by interacting with an environment. The agent performs actions and receives rewards or punishment, and it learns to choose actions that maximize the total reward over time.

Basic component.

- 1/ Agent → A learner or decision maker.
- 2/ Environment → The place where the agent operates.

- 3] Action The choices made by the agent.
- 4] Reward Feedback given to the agent for its actions. (positive or negative)
- 5] State The current situation of the environment.

Working of Reinforcement Learning.

1. The agent observes the current state of the environment.
2. It choose an action
3. The environment responds with a reward or penalty.
4. The agents updates its strategy to maximize future rewards.

Example.

A robot learning to walk.

- If it moves correctly, it gets a positive reward.
- If it falls, it get negative reward.
- Over time, the robot learns the best way to walk.

Application

- Robotics
- Game playing (like chess or video games)
- Self-driving cars.
- Recommendation systems.

Advantages

- Learns from interaction with the environment.
- Can solve complex decision-making problems.
- Improves performance over time.

Disadvantages

- Requires a large amount of training time
- Designing the reward system can be difficult
- Sometimes it may learn inefficient behaviors

Bellman Equation.

The Bellman Equation is a fundamental equation in Reinforcement learning (RL) that expresses the value of a state as the immediate reward plus the discounted value of future ~~rewards~~ states.

State Value Function.

$$V(s) = R(s) + \gamma \sum_{s'} P(s'|s) V(s')$$

Where .

- $V(s)$:- Value of state s
- $R(s)$:- Immediate reward
- γ = Discount Factor (0-1)
- $P(s'|s)$ = Probability of moving to next state s' .

Meaning $\Rightarrow$ Value of Current state = Immediate Reward + Discounted Future Reward.

The Bellman equation breaks a complex problem into smaller subproblems, making it easier to find the optimal policy.

Importance.

- Foundation of Reinforcement learning
- Used in Value Iteration and Policy Iteration.
- Helps an agent find the optimal strategy

Value Iteration & Policy Iteration

2024

Both are methods used to solve a Markov Decision Process (MDP) and find the optimal policy

1. Value Iteration

Value Iteration repeatedly updates the value of each state until the values converge

Steps

1. Initialize all state values.
2. Update values using the Bellman equation.
3. Repeat until values stop changing significantly
4. Derive the optimal policy from the final values

Advantages.

- Simple to implement
- Find the optimal policy

Disadvantages.

- May requires many iterations.

2. Policy Iteration

Policy Iteration directly improves a policy until it becomes optimal.

Two main Points

- a) Policy Evaluation:- calculate the value of states under the current policy.
 - b) Policy Improvement:- choose better actions based on the computed values.
- Repeat these two steps until the policy no longer changes.

steps

1. Start with a random policy
2. Evaluate the policy
3. Improve the policy
4. Repeat until convergence.

Advantages

usually converges in fewer iterations
often faster than value iteration

Disadvantages.

- Each iteration can be computationally expensive

Difference

Value Iteration

- 1 updates values directly
- 2 One step (value update) repeated
- 3 More Iteration
- 4 Simpler

Policy Iteration

- 1 updates policy directly
- 2 Two steps:- evaluation + improvement
- 3 fewer Iteration
- 4 Slightly more complex

Actor-Critic Model 2024

A Actor-Critic Model is a Reinforcement Learning (RL) method that combines Policy-Based Learning and Value-Based Learning.

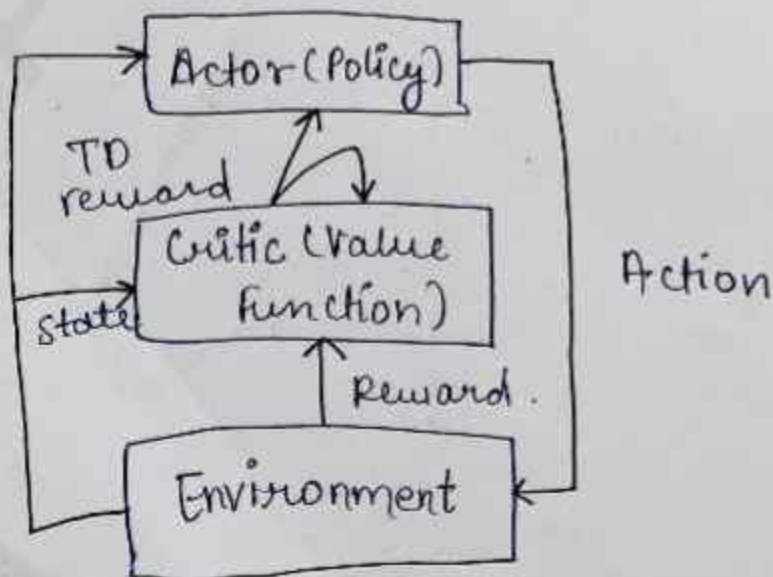
It has two components.

1. Actor:-

- Decides which action to take
- Learns the policy $\pi(a|s)$.
- Interacts with the environment.

2. Critic:-

- Evaluates the action chosen by the actor.
- Estimates the value function $V(s)$.
- Provides feedback to improve the actor.



Steps

1. Actor observes the current state
2. Actor selects an action
3. Environment returns reward and next state.
4. Critic evaluates how good the action was.
5. Critic sends feedback.
6. Actor updates its policy and improves future actions.

Example.

Imagine a robot learning to walk.

- Actor - Decides how to move its legs
- Critic - Checks whether the movement improved balance and speed.
- if good → positive feedback
- if bad → negative feedback

Over time, the robot learns to walk efficiently

Advantages

learns faster than many basic RL methods.

Handles large state spaces

combines strength of policy and value methods

suitable for continuous actions.

Application

- Robotics
- self driving cars
- Game Playing
- Resource allocation

Q-Learning

2025

Q-Learning is a model-free Reinforcement learning algorithm that learns the best action to take in each state by estimating Q-values (Quality Values).

A Q-value represents the expected future reward of taking an action in a particular state.

Q-Value

$$Q(s, a).$$

where

- s = state
- a = Action
- $Q(s, a)$ = Expected reward for taking action a in state s .

Q-Learning update Equation.

$$Q(s, a) = Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Where

- α (alpha) = Learning rate
- γ (gamma) = Discount factor
- r = Immediate reward
- s' = Next state
- $\max Q(s', a') =$ Maximum future reward.

Working steps.

1. Observe current state s .
2. Choose an action a .
3. Receive reward and next state (s').
4. Update the Q -value using the Q -learning equation.
5. Repeat until the optimal policy is learned.

Q-Table Example.

State	Left	Right
s_1	2	5
s_2	3	8

The agent chooses the action with the highest Q-value.

Example.

In S1, choose Right because 572.

Advantages.

- Simple and easy to implement
- Does not require a model of the environment
- Learns the optimal policy through experience.

Disadvantages.

- Large state spaces require large Q-tables.
- Learning can be slow for complex problems.

Application.

- Robotics
- Game Playing
- Path Finding
- Resource Management.

SARSA

(State - Action - Reward - State - Action)

SARSA is a model-free Reinforcement Learning algorithm that learns the action value function $Q(s,a)$ based on the action actually taken by the agent

The name SARSA comes from:

$S \rightarrow A \rightarrow R \rightarrow S \rightarrow A$

- State
- Action
- Reward
- Next state
- Next Action

working steps

1. Observe current state s .
2. Choose action a
3. Take action and receive reward r .
4. Observe next state s'
5. Choose next action a'
6. Update Q -value
7. Repeat

2024

Q Describe the significance of n-gram precision and brevity penalty in BLEU score calculation.

Ans → BLEU - (Bilingual Evaluation Understudy) is a metric used to evaluate the quality of machine-translated text by comparing it with one or more reference translations.

N-gram Precision → N-gram precision measures how many words or sequences of words (n-gram) in machine-generated translation match those in the reference translation.

- 1-gram (Unigram) - Checks individual words
- 2-gram (Bigram) - Checks pair of consecutive words.

- 3-gram (Trigram) - check sequences of three words
- 4-gram. Check sequences of four words.

Importance →

- Evaluates the accuracy of translated words and phrases.
- Higher-order n -grams capture fluency and correct word order.
- Prevents a system from getting a high score by simply using correct words in the wrong sequence.

Example: Reference: - The cat is on the mat

Candidate: - The cat is on mat.

- Many words match, so unigram precision is high, but some higher-order n -grams are missing.

• Brevity Penalty (BP) :- A translation

system could achieve high precision by producing very short translations containing only common matching words. To avoid this.

BLEU uses a Brevity Penalty.

Formula

$$BP = \begin{cases} 1 & c > r \\ e^{(1-r/c)} & c \leq r \end{cases}$$

where c = length of candidate translation
 r = length of reference translation

Importance

Penalize translations that are too short

Encourages complete translations

Balance precision with adequate translation length.

Significance in BLEU score

- N-gram precision measures translation accuracy and fluency.
- Brevity Penalty ensures the translation is not excessively short
- Together, they provide a balanced evaluation of machine translation quality